Deliverable 5: AI Partnership Log

A candid record of how AI was used building this

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

AI Partnership Log

Which tools, for what

Claude Code (Sonnet 5) was the primary collaborator for the whole build: architecture, the data generator, feature engineering, the anomaly detector, the FastAPI backend, the React frontend, the AI copilot layer, the Playwright suite, CI, and first drafts of every doc. I directed it, reviewed everything it produced, and pushed back when something didn't sit right. Separately, Claude Haiku 4.5 runs inside the product itself as the fleet copilot. That's a different relationship, it's a component I built and had to treat like any other dependency, not trust because it shares a name with my coding assistant.

Overrides, where I didn't just take what it gave me

Accepting the first "it works" without asking what it cost. First pass at the alert threshold used the 99.5th percentile of normal scores, conservative sounding, and it caught zero of eleven real failures. Instead of taking the next fix at face value, I had it sweep the full threshold range and show me the actual detection rate versus false alarm rate at every point. Landed on 97th percentile, full detection, because I saw the tradeoff, not because it was the first setting with a good headline number.

Documentation voice. Claude's default for technical docs is polished, third person, corporate. I asked for first person, what I actually tried, what didn't work. This assignment scores the log on candor, and generic voice would have flattened real back and forth into something nobody actually wrote. I also had to push a second time on em dashes and hyphen as punctuation showing up constantly, a small tic that reads as machine written the moment you notice it.

Leaving Streamlit for a real frontend. The dashboard worked but looked like a prototype. I told it to drop Streamlit entirely and rebuild in React and TypeScript with a proper backend, not restyle the existing script, which meant redoing the frontend, the test suite, and CI from a working baseline. Worth it: a shift supervisor's tool needs to look trustworthy, not like a demo.

Not trusting a flaky looking test as probably fine. One Playwright test failed intermittently after the React rebuild. I'd already made a legitimate unrelated fix nearby and nearly called it solved. Instead I wrote a standalone script that clicked the same element in a real browser and printed the actual state, no test framework involved, which proved the click itself worked. Only then did I treat the remaining flakiness as real environmental noise and fix the actual cause: timeouts and retries in the test config, not a suppressed assertion.

Publishing the confidential prompt. I told it to sync all the setup to the public repo, meaning everything, including the take home prompt itself. It flagged, unprompted, that the assignment says not to post that document publicly, and asked before pushing anything. I hadn't thought about the prompt file sitting next to the code. This is the one override that came from the AI catching me, not the other way around.

Confidently wrong, and how it got caught

The rolling mean baseline bug. For a stand genuinely mid failure, the copilot said coolant pressure was up. Backwards, the failure signature is pressure dropping. It wasn't hallucinating a number, it correctly reported the z score it was handed, but that z score was computed against a 60 minute rolling mean that was itself mid collapse during the fault, so a noisy uptick inside an ongoing drop can register as above normal. I caught it by reading the output for

internal consistency, "up" didn't match the failure signature I knew, and fixed it by comparing against a stable, whole period baseline instead.

A sub agent reported success on work it never did. I delegated a mechanical cleanup, rewriting dash punctuation across 18 files, to a background sub agent so it wouldn't eat the main session's context. It came back marked completed with a written summary claiming the work was done. When I actually checked by re searching the codebase, every instance was still there. It had gotten confused partway through and reported the confusion as a finished task. I only caught it because I check the files, not the status message, and did the rewrite myself afterward.

What I won't let it decide alone

Whether an anomaly triggers a real world action, the dashboard explains and suggests, it never shuts anything down or dispatches anyone. Where the actual alert threshold sits, it can compute the tradeoff curve but not how much false alarm fatigue a real shift crew would tolerate before they start ignoring the tool. Whether something is safe to publish, the confidentiality catch above is the clearest example, I take the flag seriously but the push decision is mine. Whether a task actually got done, a status of completed is a claim, not a fact, I check the artifact.

Where it was 10x, where it slowed me down

10x on sheer volume: a synthetic data generator with a realistic failure ramp, a feature pipeline, a trained and evaluated detector, a backend, a frontend, an LLM explanation layer, a test suite, a CI workflow, and this set of docs, in far less time than doing the typing myself across that many different kinds of work.

Slower in two places. The editor's type checker kept flagging pandas and scikit-learn calls as errors that weren't real, stale stub noise that needed a second look every time to confirm it was nothing. And the sub agent failure above cost real time, a delegated task that silently didn't happen is worse than one that visibly fails, because it looks finished until you check.

Full detail on every override, bug, and addendum along the way is in `docs/ai-partnership-log.md` in the repository.